

DEVOPS CASE STUDY REPORT

Vị trí ứng tuyển: DevOps Engineer

Người thực hiện: Lê Vàng Dũ

LỰA CHỌN TASK VÀ PHẠM VI THỰC HIỆN (SCOPE OF WORK)

Để hoàn thành trọn vẹn yêu cầu của bài Practical Exercise, báo cáo này tập trung giải quyết và phân tích sâu các Task sau:

- Task 0 – Kubernetes Deployment (BẮT BUỘC):** Sử dụng Helm để triển khai ứng dụng lên cụm K8s local, cấu hình tài nguyên cơ bản (Deployment, Service), quản lý biến môi trường và thiết lập chiến lược phát hành.
- Task 1 – Jenkins CI/CD Pipeline:** Thiết kế và xây dựng chu trình tự động hóa hoàn chỉnh từ khâu Checkout, Test, Build Image, Push Image đến Deploy.
- Task 4 - Pipeline Optimization.**
- Task 5 - Production Incident.**

Chiến lược tích hợp Task 0 và Task 1:

Trong quá trình thực hiện, **Task 0 (Kubernetes Deployment)** và **Task 1 (Jenkins CI/CD Pipeline)** được gộp chung hoàn toàn vào một luồng xử lý tự động hóa duy nhất. Việc tích hợp này mang lại hai lợi ích cốt lõi:

- Thuận tiện thực hành:** Tránh việc vận hành rời rạc các script thủ công. Jenkins Pipeline (Task 1) sẽ đóng vai trò là "bộ não" điều khiển và thực thi trực tiếp việc đóng gói, cập nhật và quản lý vòng đời của các tài nguyên trên Kubernetes Cluster (Task 0).
- Tối ưu hóa phương án giải quyết:** Giúp kiểm thử trực tiếp tính đúng đắn của các chiến lược như Rolling Update, Auto-Rollback và quản lý biến môi trường theo thời gian thực (Real-time). Khi có lỗi xảy ra ở bất kỳ công đoạn nào (từ code cho đến hạ tầng K8s), Pipeline sẽ lập tức ghi nhận và đưa ra phản hồi chính xác.

1. GIẢI THÍCH THIẾT KẾ PIPELINE TÍCH HỢP (PIPELINE DESIGN & ARCHITECTURE)

Hệ thống CI/CD tích hợp (Task 1 + Task 0) được thiết kế dựa trên nguyên lý **Fail-Fast** và kiến trúc **Artifact-Based Promotion**. Luồng xử lý đảm bảo mã nguồn được kiểm tra nghiêm ngặt trước khi được đóng gói thành một Artifact bất biến để triển khai lên Kubernetes thông qua Helm Chart.

Các giai đoạn chính trong Jenkinsfile:

1. Giai đoạn 0: Checkout SCM

- **Mục tiêu:** Tải (pull) toàn bộ mã nguồn mới nhất từ hệ thống quản lý phiên bản (Source Control Management - cụ thể là GitHub) về workspace của Jenkins Agent trước khi thực thi bất kỳ tác vụ nào.
- **Cơ chế hoạt động:** Trong dự án này, Jenkins được thiết lập sử dụng tính năng **Pipeline script from SCM**. Khi Pipeline được kích hoạt, Jenkins sẽ tự động xác thực với Git Repository và sao chép toàn bộ thư mục dự án (bao gồm mã nguồn Node.js, thư mục cấu hình Helm và file định nghĩa **Jenkinsfile** từ nhánh **main** xuống máy chủ.
- **Ý nghĩa kiến trúc:** Giai đoạn này thực thi nghiêm ngặt nguyên tắc **Single Source of Truth**. Mọi thay đổi mã nguồn hay hạ tầng đều bắt buộc phải được commit lên Git. Nó ngăn chặn hoàn toàn thói quen sửa lỗi trực tiếp trên server, đảm bảo tính minh bạch, nhất quán và khả năng truy vết lịch sử cho mọi phiên bản triển khai.

2. Giai đoạn 1: Code Analysis & Unit Test (Kiểm thử & Phân tích)

- **Mục tiêu:** Thực thi Linting và Unit Test nhằm phát hiện sớm các lỗi logic cơ bản.
- **Cơ chế cách ly:** Các câu lệnh được thực thi hoàn toàn biệt lập bên trong một Node.js container tạm thời thông qua cơ chế sao chép vật lý, loại bỏ các xung đột về môi trường của Agent. Giai đoạn này sử dụng tham số động **TEST_MODE** để giả lập các kịch bản Pass/Fail phục vụ quá trình demo.



```
devops-case-study / demo-app / package.json
duleworking1103 Create package.json 2345fb8 · 1 hour ago
Code Blame 10 lines (10 loc) · 322 Bytes Raw
1 {
2   "name": "devops-case-study-app",
3   "version": "1.0.0",
4   "main": "server.js",
5   "scripts": {
6     "start": "node server.js",
7     "lint": "echo '🟢 Code style check passed!'",
8     "test": "if [ \"$TEST_MODE\" = \"fail\" ]; then echo '🔴 Mock Test Failed!'; exit 1; else echo '🟢 Mock Test Passed!'; exit 0; fi"
9   }
10 }
```

3. Giai đoạn 2: Build & Push Artifact (Đóng gói & Phân phối)

- **Mục tiêu:** Khi mã nguồn vượt qua các bài kiểm tra, hệ thống tiến hành build Docker Image chính thức từ **demo-app/Dockerfile**.
- **Quản lý bảo mật:** Sử dụng block **withCredentials** bảo mật của Jenkins để đăng nhập vào Docker Registry và đẩy Image lên với một Tag phiên bản duy nhất (**IMAGE_TAG**).

4. Giai đoạn 3: Deploy to DEV (Triển khai môi trường phát triển trên K8s)

- **Mục tiêu:** Tự động triển khai phiên bản mới nhất lên môi trường DEV trên cụm Kubernetes local bằng Helm Chart, truyền động các biến cấu hình (**DEV_API_URL_SECRET**) từ Jenkins Credentials.

5. Giai đoạn 4: Approval for PROD (Quality Gate)

- **Mục tiêu:** Hệ thống tạm dừng và yêu cầu quản trị viên xác nhận hệ thống trên DEV hoạt động ổn định trước khi cho phép mã nguồn tiến lên môi trường sản xuất.

6. Giai đoạn 5: Deploy to PROD & Manual Rollback.

- **Mục tiêu:** Phát hành ứng dụng lên Production K8s Cluster. Ngay sau khi deploy thành công, hệ thống mở ra một cửa sổ thời gian (5 phút) cho phép vận hành viên thực hiện kịch bản "Rollback khẩn cấp" bằng tay nếu phát hiện lỗi nghiệp vụ thực tế phát sinh.

2. CÁC QUYẾT ĐỊNH THIẾT KẾ KỸ THUẬT (DESIGN DECISIONS - TASK 0 & TASK 1)

2.1. Lựa chọn Công cụ Triển khai: Helm Chart vs. Kubectl Apply

Báo cáo này lựa chọn **Helm Chart** làm công cụ quản lý phân phối chính trên Kubernetes thay vì sử dụng các file manifest tĩnh với `kubectl apply`.

- **Quản lý cấu hình động:** Helm cho phép đóng gói toàn bộ các tài nguyên cơ bản (Deployment, Service) vào một cấu trúc Chart duy nhất và thay đổi các tham số (như `image.tag`, `environment`, `secretApiUrl`) một cách linh hoạt theo từng môi trường thông qua cờ `--set`.
- **Quản lý vòng đời Release:** Helm tự động lưu lại lịch sử các lần deploy dưới dạng các Revision độc lập. Điều này giúp loại bỏ việc quản lý thủ công hàng chục file YAML tĩnh và giảm thiểu rủi ro sai sót con người.

2.2. Chiến lược Phục hồi Hệ thống (Rollback Strategy)

Hệ thống tích hợp được thiết kế với hai lớp bảo vệ toàn diện chống lại các sự cố triển khai:

- **Lớp 1 - Auto-Rollback:** Trong hàm `deployAndVerify`, lệnh Helm được chạy với các tham số `--wait --timeout 2m`. Khi ứng dụng gặp lỗi crash-loop, sai biến môi trường, hoặc không vượt qua `Readiness Probe` của Kubernetes trong vòng 2 phút, khối `catch` trong Jenkinsfile sẽ lập tức kích hoạt lệnh `helm rollback <release-name> 0` để đưa cụm K8s về trạng thái ổn định gần nhất trước đó.
- **Lớp 2 - Manual Rollback (Lỗi logic nghiệp vụ):** Đối với các lỗi logic mà K8s vẫn ghi nhận Pod ở trạng thái "Healthy", khối lệnh `timeout(time: 5, unit: 'MINUTES')` kết hợp với `input` trên Pipeline cho phép vận hành viên chủ động nhấn nút hủy bỏ, thực thi lệnh `helm rollback` ngay lập tức để hạ cấp ứng dụng về phiên bản cũ an toàn.

2.3. Giải pháp Mở rộng Ứng dụng (Scaling Application)

Để đảm bảo khả năng chịu tải và tính sẵn sàng cao, hệ thống áp dụng hai cơ chế scale:

- **Scale tĩnh (Manual Scaling):** Định nghĩa tham số `replicaCount` trong file `values.yaml` của Helm Chart để duy trì tối thiểu nhiều Pod chạy song song, phân phối tải thông qua Kubernetes Service.
- **Scale động (Autoscaling):** Sử dụng cấu hình `Horizontal Pod Autoscaler` (HPA) giám sát các chỉ số tài nguyên của Pod. Hệ thống sẽ tự động tăng số lượng Pod lên khi mức độ tiêu thụ CPU hoặc RAM trung bình vượt ngưỡng 75%, và tự động giảm số lượng (Scale down) khi lượng truy cập thấp để tối ưu chi phí hạ tầng.

2.4. Các vấn đề thường gặp khi triển khai Kubernetes.

Trong quá trình vận hành hệ thống Kubernetes thực tế, việc đối mặt với các lỗi trạng thái của Pod là không thể tránh khỏi. Dưới đây là các vấn đề phổ biến nhất và phương pháp giải quyết (Troubleshooting) được áp dụng:

1. Lỗi `CrashLoopBackOff`

- **Hiện tượng:** Pod được tạo ra nhưng container bên trong khởi động lên rồi lập tức bị sập (crash), quá trình này lặp đi lặp lại liên tục.
- **Nguyên nhân cốt lõi:** Thường do lỗi mã nguồn (ứng dụng văng Exception khi chạy), sai câu lệnh khởi động (CMD/Entrypoint), hoặc ứng dụng không tìm thấy biến môi trường/file cấu hình (ConfigMap/Secret) cần thiết.
- **Cách xử lý:** Sử dụng lệnh `kubectl logs <tên-pod> --previous` để đọc stacktrace báo lỗi của container ngay trước khi nó bị sập, từ đó vá lỗi ở tầng ứng dụng hoặc cập nhật lại Helm values.

2. Lỗi `ImagePullBackOff` hoặc `ErrImagePull`

- **Hiện tượng:** K8s không thể tải được Docker Image từ Registry về Node để chạy.
- **Nguyên nhân cốt lõi:** Sai tên Image, sai Tag (ví dụ Tag không tồn tại trên Docker Hub), hoặc Cluster không có quyền truy cập vào Private Registry.
- **Cách xử lý:** Sử dụng lệnh `kubectl describe pod <tên-pod>` và cuộn xuống phần **Events** để xem chi tiết lỗi pull. Nếu là lỗi xác thực, cần tạo `imagePullSecrets` và khai báo vào file Deployment.

3. Lỗi `OOMKilled` (Out of Memory)

- **Hiện tượng:** Pod đột ngột bị K8s cưỡng chế tắt (Terminated).

- **Nguyên nhân cốt lõi:** Ứng dụng bên trong container tiêu thụ bộ nhớ RAM vượt quá giới hạn cứng (`resources.limits.memory`) được khai báo trong file cấu hình.
- **Cách xử lý:** Phân tích mã nguồn để xử lý hiện tượng rò rỉ bộ nhớ (Memory Leak), hoặc điều chỉnh tăng mức giới hạn RAM trong file `values.yaml` của Helm cho phù hợp với nhu cầu thực tế của ứng dụng.

4. Trạng thái Pod bị treo ở **Pending**

- **Hiện tượng:** Pod đã được khai báo nhưng không thể lên lịch (schedule) để chạy trên bất kỳ Node nào.
- **Nguyên nhân cốt lõi:** Cụm Cluster đã cạn kiệt tài nguyên (CPU/RAM không đủ đáp ứng `requests` của Pod), hoặc lỗi gán nhãn Node (Node Selector/Taints) không khớp.
- **Cách xử lý:** Chạy `kubectl describe pod <tên-pod>` để xem thông báo từ Scheduler. Nếu thực sự thiếu tài nguyên, cần Scale up cụm K8s hoặc tối ưu giảm mức yêu cầu tài nguyên của ứng dụng.

5. Lỗi mất kết nối Service (Network Routing Issues)

- **Hiện tượng:** Pod chạy `Running`, hiển thị log bình thường nhưng người dùng hoặc các ứng dụng khác không thể gọi API tới nó.
- **Nguyên nhân cốt lõi:** Khai báo sai nhãn (Labels) giữa phần `selector` của Service và `template` của Deployment, hoặc ánh xạ sai cổng (`targetPort` của Service không khớp với port mà container đang lắng nghe).
- **Cách xử lý:** Kiểm tra kết nối chéo bằng cách dùng `kubectl get endpoints <tên-service>`. Nếu cột Endpoints báo `<none>`, chắc chắn cấu hình selector đang bị sai lệch.

3. GIẢI THÍCH MÃ NGUỒN ỨNG DỤNG VÀ KỊCH BẢN MOCK TEST (APPLICATION & MOCK TEST DESIGN)

Ứng dụng demo được viết bằng Node.js thuần (sử dụng module `http` mặc định) để tối ưu hóa thời gian build và thu gọn dung lượng Docker Image (không cần cài đặt thêm framework bên thứ ba như Express). Mã nguồn được chủ đích thiết kế nhằm phục vụ việc kiểm thử các tính năng cốt lõi của Kubernetes và CI/CD.

3.1. Thiết kế Ứng dụng (server.js) tuân thủ 12-Factor App

Mã nguồn ứng dụng được xây dựng theo chuẩn 12-Factor App, đặc biệt tập trung vào yếu tố "Lưu trữ cấu hình trong biến môi trường" (Config in environment).

- **Quản lý cấu hình động (Dynamic Configuration):** Ứng dụng không hard-code các thông tin nhạy cảm hay thông tin môi trường. Thay vào đó, nó nhận các giá trị từ K8s thông qua biến môi trường (`process.env`), bao gồm:

`APP_ENV`, `SERVICE_NAME`, `APP_VERSION`, và `SECRET_API_URL`. Khi gọi vào ứng dụng, nó sẽ trả về chuỗi JSON chứa các thông số này, giúp dễ dàng xác minh rằng Helm đã inject đúng cấu hình cho từng môi trường (DEV/PROD).

- **Kubernetes Probes Endpoint (`/health`):** Ứng dụng phơi bày một endpoint chuyên biệt tại đường dẫn `/health`. Đây là "nhịp tim" của ứng dụng để Kubernetes thực hiện các bài kiểm tra `ReadinessProbe` và `LivenessProbe`.
 - **Trạng thái bình thường:** Trả về HTTP Status `200 OK`.
 - **Giả lập sự cố (Mock Failure):** Bằng cách truyền vào biến môi trường `APP_MODE=broken`, endpoint này sẽ cố tình trả về HTTP Status `500 (Internal Server Error)`. Kịch bản này được sử dụng để đánh lừa Kubernetes rằng ứng dụng đang bị crash, từ đó kích hoạt cơ chế Auto-Rollback (chặn không cho release bản lỗi) mà chúng ta đã thiết kế trong Helm và Jenkins.

3.2. Kịch bản Mock Test và Cơ chế Exit Code (package.json)

Để phục vụ việc chạy luồng CI/CD mà không cần thiết lập một hệ thống Unit Test phức tạp (như Jest hay Mocha), dự án sử dụng các Mock Script tận dụng trực tiếp tính năng Exit Code của hệ điều hành Linux.

- **Script `lint` (Kiểm tra cú pháp):** Được giả lập bằng lệnh `echo`. Trong thực tế, đây sẽ là nơi gọi ESLint.
- **Script `test` (Kiểm thử tự động & Fail-fast):** Đoạn script được viết bằng Shell: `if [ "$TEST_MODE" = "fail" ]; then echo '❌ Mock Test Failed!'; exit 1; else echo '✅ Mock Test Passed!'; exit 0; fi`
 - Khi `TEST_MODE=pass`: Script gọi lệnh `exit 0`. Trong chuẩn POSIX, số 0 báo hiệu chương trình chạy thành công. Jenkins nhận được tín hiệu này sẽ tiếp tục chạy các stage tiếp theo (Build & Push).
 - Khi `TEST_MODE=fail`: Script gọi lệnh `exit 1`. Bất kỳ số nào khác 0 đều báo hiệu có lỗi (Fatal Error). Jenkins khi bắt được `exit 1` sẽ lập tức đánh dấu Pipeline là thất bại (Đỏ), hủy bỏ việc đóng gói Docker Image và dừng toàn bộ luồng triển khai. Đây chính là minh chứng rõ nét nhất cho nguyên lý Fail-fast trong hệ thống CI/CD.

4. HƯỚNG DẪN CHẠY DEMO

4.1. Prerequisites

- Docker & Docker Compose.
- Kubernetes Cluster local (K3s).
- Công cụ dòng lệnh `kubectl` và `helm`.

4.2. Khởi chạy môi trường Jenkins local bằng Docker Compose

1. Di chuyển đến thư mục gốc của dự án, chạy lệnh: `docker-compose up -d`.
2. Truy cập giao diện web của Jenkins tại địa chỉ: `http://localhost:8080`.

4.3. Thiết lập thông tin bảo mật (Jenkins Credentials)

Khai báo 3 khóa bảo mật trong mục *Manage Jenkins > Credentials*:

- `DOCKER_REGISTRY_CREDS`: Loại *Username with password*, chứa thông tin tài khoản Docker Hub.

Update credential

Scope ?
Global (Jenkins, nodes, items, all child items, etc) ▾

Username
dule1103

☐ Treat username as secret ?

Password
Concealed Change Password

ID ?
DOCKER_REGISTRY_CREDS

Description ?
Dockerhub cred

Save

- **DEV_API_URL_SECRET**: Loại *Secret text*, chứa thông tin URL giả lập cho môi trường DEV.

Update credential

×

Scope ?
Global (Jenkins, nodes, items, all child items, etc) ▼

Secret
Concealed Change Password

ID ?
DEV_API_URL_SECRET

Description ?
Secret for Dev Env

Save

- **PROD_API_URL_SECRET**: Loại *Secret text*, chứa thông tin URL giả lập cho môi trường PRODUCTION.

Update credential

×

Scope ?
Global (Jenkins, nodes, items, all child items, etc) ▼

Secret
Concealed Change Password

ID ?
PROD_API_URL_SECRET

Description ?
Secret for Prod Env

Save

4.4. Cấu hình và Chạy Pipeline

1. Tạo một **Pipeline** item mới trên Jenkins, cấu hình trở về Public Github Repository của dự án (nhánh **main**).

Definition

Pipeline script from SCM

SCM ?

Git

Repositories ?

Repository URL ?

https://github.com/duleworking1103/devops-case-study.git

Credentials ?

- none -

+ Add

Advanced

+ Add Repository

Branches to build ?

Branch Specifier (blank for 'any') ?

*/main

+ Add Branch

Repository browser ?

(Auto)

Additional Behaviours

Save Apply

2. Nhấn **Build Now** lần đầu để đồng bộ tham số đầu vào.

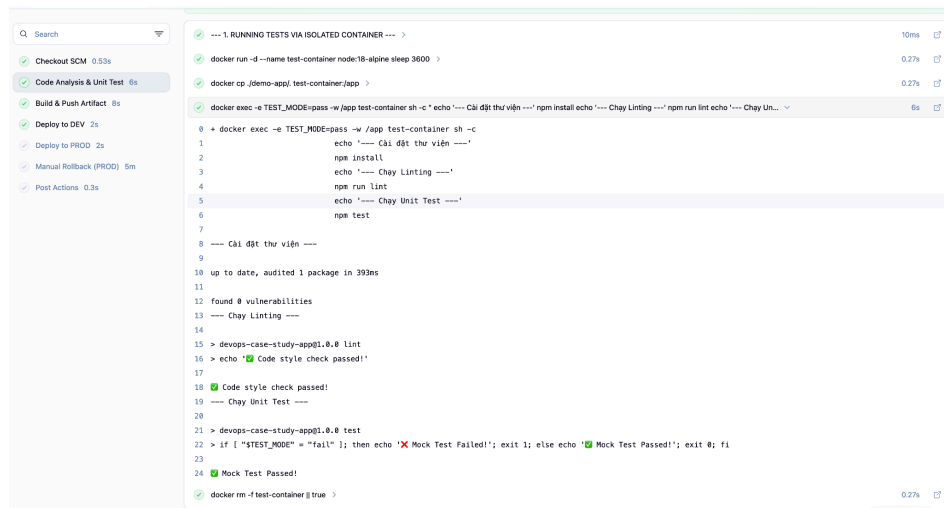
5. KỊCH BẢN DEMO THỰC NGHIỆM (PRACTICAL DEMO SCENARIOS)

Để chứng minh tính hiệu quả và sự mạnh mẽ của hệ thống CI/CD tích hợp Kubernetes đã thiết kế, báo cáo này thực hiện 3 kịch bản demo kiểm thử các rủi ro thường gặp trong quá trình vận hành thực tế.

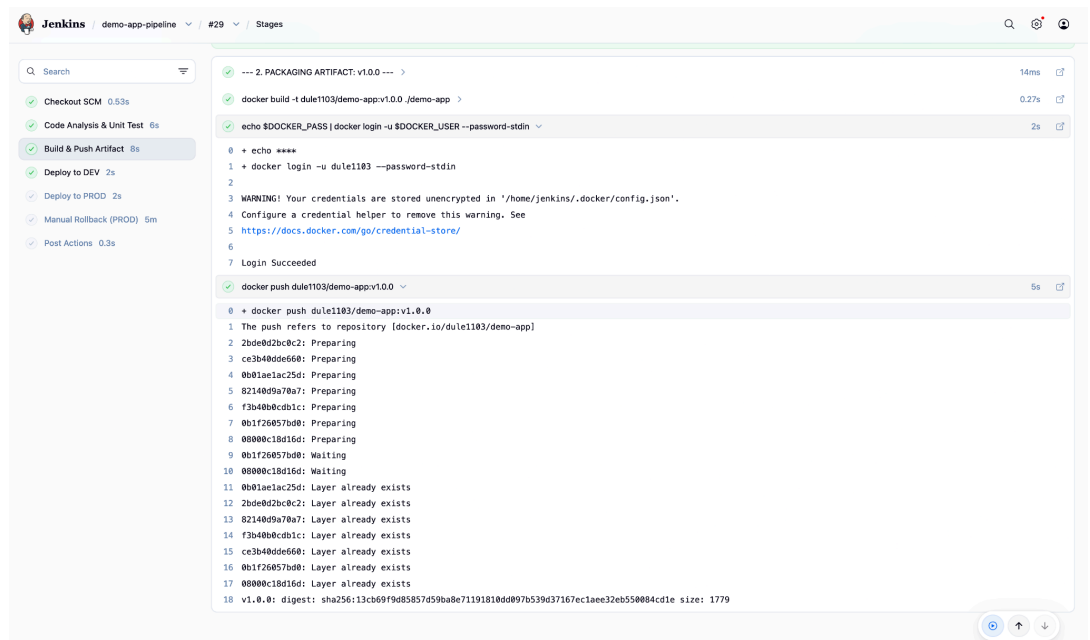
Kịch bản 1: Luồng Triển khai Chuẩn Thành công toàn diện

Mục tiêu: Chứng minh mã nguồn an toàn được đóng gói và phân phối trơn tru qua các môi trường, dưới sự kiểm soát của con người.

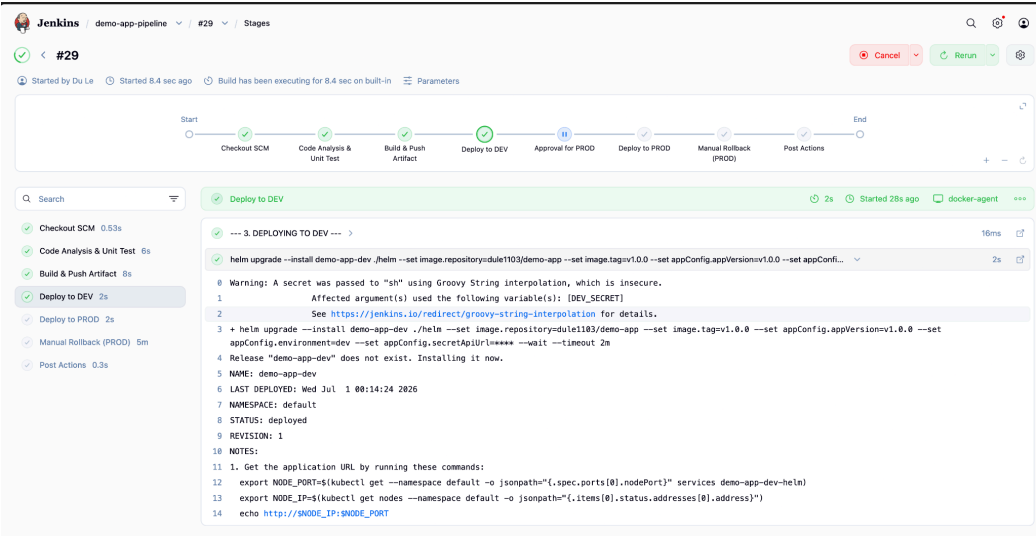
- **Tham số thực thi:** `IMAGE_TAG = v1.0.0`, `TEST_MODE = pass`.
- **Diễn biến:**
 1. Pipeline vượt qua bài mock test



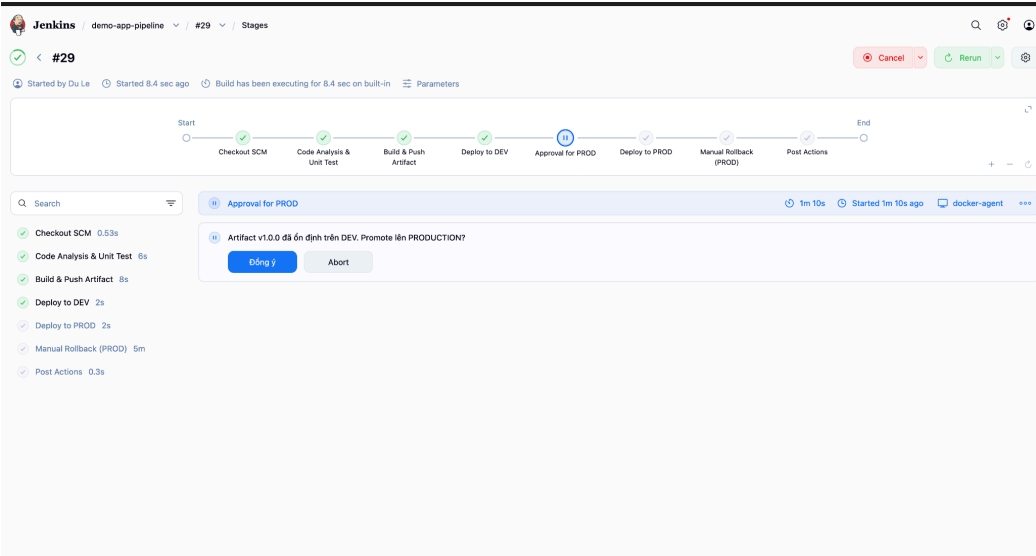
2. Docker Image được đẩy lên Registry.



3. Triển khai tự động lên cụm K8s môi trường DEV.



4. Người quản trị xác nhận tính ổn định và nhấn nút "Đồng ý Promote".



5. Hệ thống triển khai thành công lên môi trường PROD và hoàn tất sau 5 phút chờ xác nhận.

```

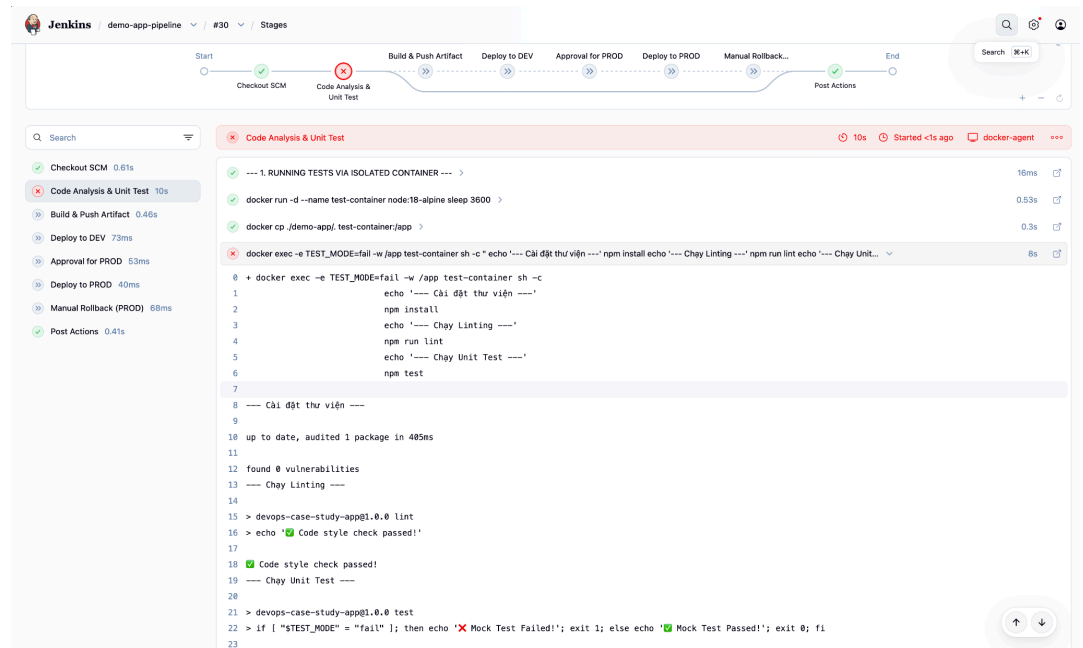
root@dev-ubuntu:/home/jenkins/agent/workspace# kubectl get pods
No resources found in default namespace.
root@dev-ubuntu:/home/jenkins/agent/workspace# kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
demo-app-dev-helm-69c6656d4-blp85   1/1     Running   0           3m30s
demo-app-dev-helm-69c6656d4-kk84c   1/1     Running   0           3m30s
root@dev-ubuntu:/home/jenkins/agent/workspace# kubectl get svc demo-app-dev-helm
NAME                                TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
demo-app-dev-helm                  NodePort    10.43.216.251 <none>        3000:32137/TCP  3m53s
root@dev-ubuntu:/home/jenkins/agent/workspace# curl http://localhost:32137
{
  "message": "DevOps Case Study App is Running!",
  "config": {
    "Environment": "dev",
    "service": "Unknown Service",
    "version": "v1.0.0",
    "secret_api": "https://dev.secret-api-key.com/"
  }
}
root@dev-ubuntu:/home/jenkins/agent/workspace# kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
demo-app-dev-helm-69c6656d4-blp85   1/1     Running   0           17m
demo-app-dev-helm-69c6656d4-kk84c   1/1     Running   0           17m
demo-app-production-helm-85684df88c-6ld6m 1/1     Running   0           4m52s
demo-app-production-helm-85684df88c-fzj8b 1/1     Running   0           4m52s
root@dev-ubuntu:/home/jenkins/agent/workspace# kubectl get svc demo-app-production-helm
NAME                                TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
demo-app-production-helm          NodePort    10.43.193.16 <none>        3000:30486/TCP  5m14s
root@dev-ubuntu:/home/jenkins/agent/workspace# curl http://localhost:30486
{
  "message": "DevOps Case Study App is Running!",
  "config": {
    "Environment": "production",
    "service": "Unknown Service",
    "version": "v1.0.0",
    "secret_api": "https://prod.secret-api-key.com"
  }
}

```

Kịch bản 2: Cơ chế Fail-Fast

Mục tiêu: Chứng minh hệ thống có khả năng tự bảo vệ, không lãng phí tài nguyên để build Image nếu mã nguồn không đạt chuẩn chất lượng (Lỗi Unit Test / Linter).

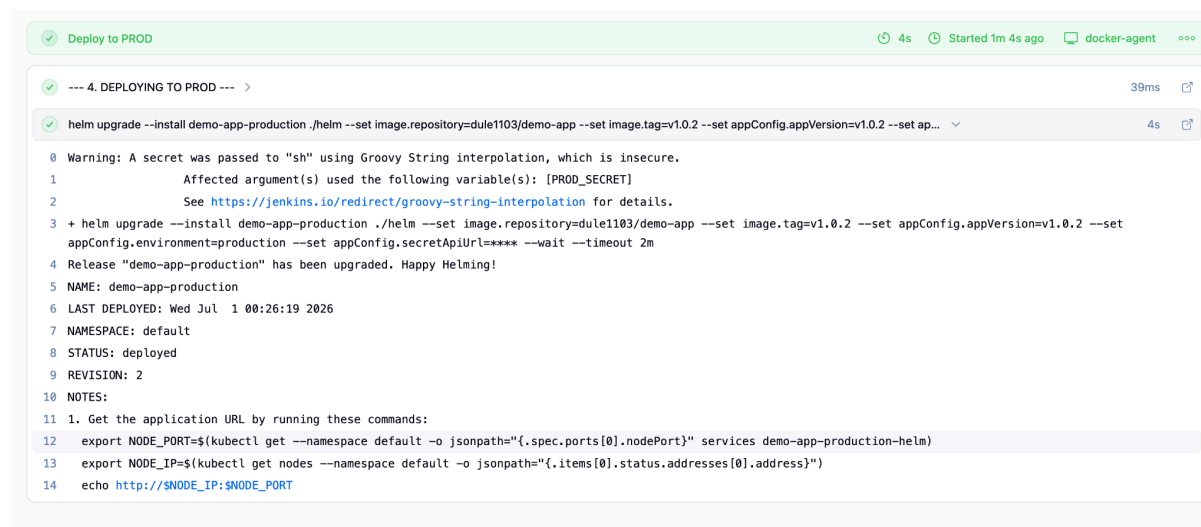
- **Tham số thực thi:** `IMAGE_TAG = v1.0.1`, `TEST_MODE = fail`.
- **Diễn biến:**
 1. Mã nguồn được kéo về và đưa vào Isolated Container để test.
 2. Phát hiện lỗi tại bước Unit Test.
 3. Pipeline ngay lập tức dừng lại (Đổi màu Đỏ).
 4. Container dùng để test được dọn dẹp sạch sẽ. Các bước Build Image và Deploy phía sau bị vô hiệu hóa hoàn toàn (Skipped).



Kịch bản 3: Kích hoạt Manual Rollback (Bảo vệ môi trường Production)

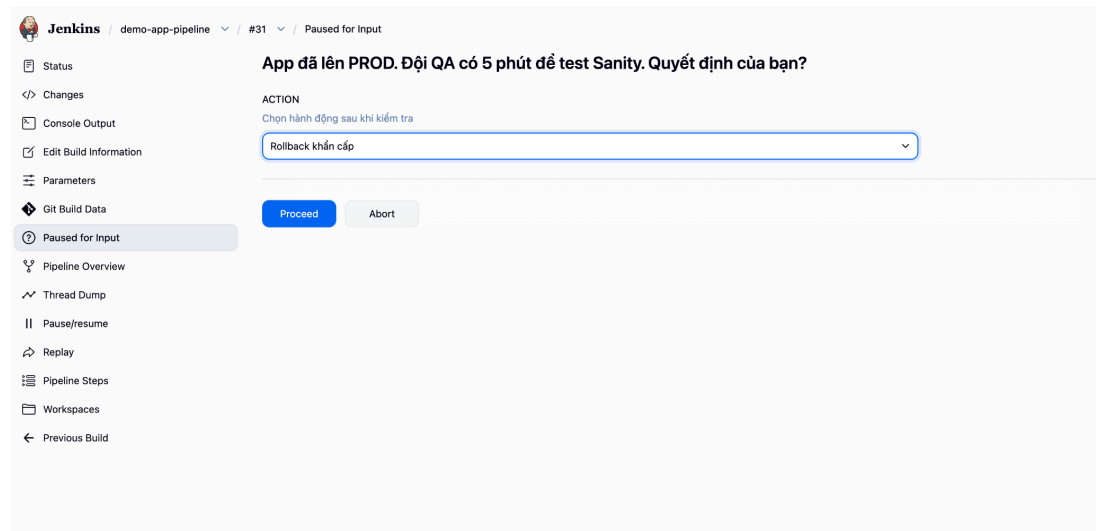
Mục tiêu: Mô phỏng kịch bản lỗi logic nghiệp vụ (Ví dụ: Giao diện bị lệch, nút thanh toán không hoạt động) mà Kubernetes Health Check không thể tự phát hiện được. Thể hiện khả năng phản ứng và khôi phục hệ thống (MTTR - Mean Time To Recovery) trong vòng vài giây.

- **Tham số thực thi:** Chạy tiếp một bản build mới (ví dụ: `IMAGE_TAG = v1.0.2`, `TEST_MODE = pass`).
- **Diễn biến:**
 1. Vì không thể giả lập được mã nguồn bị lỗi vì sử dụng mã nguồn quá đơn giản, nên sau khi deploy thành công ở Prod.



REVISION: 2

2. Can thiệp vào Jenkins, chọn hành động "Rollback khẩn cấp".



3. Lệnh `helm rollback` lập tức được thực thi, đưa hệ thống lùi về phiên bản an toàn trước đó (Rollback to 1).

```
root@dev-ubuntu:/home/jenkins/agent/workspace# helm history demo-app-production
REVISION      UPDATED              STATUS      CHART              APP VERSION      DESCRIPTION
1             Wed Jul 1 00:16:51 2026  superseded  helm-0.1.0         1.16.0           Install complete
2             Wed Jul 1 00:26:19 2026  superseded  helm-0.1.0         1.16.0           Upgrade complete
3             Wed Jul 1 00:26:44 2026  deployed   helm-0.1.0         1.16.0           Rollback to 1
```

5. TASK 4: PIPELINE OPTIMIZATION

Ngữ cảnh: Thời gian chạy Pipeline mất ~25 phút, Docker build thường xuyên thất bại và bộ phận phát triển phàn nàn về việc triển khai quá chậm.

5.1. Phân tích nguyên nhân (Root Cause Analysis)

- **Thời gian Build kéo dài (25 phút):** Do Pipeline thực hiện các tác vụ một cách tuần tự. Mỗi lần chạy đều phải tải lại toàn bộ thư viện (dependencies) từ đầu qua internet thay vì dùng cache. Dockerfile có thể chưa được tối ưu thứ tự các layer, dẫn đến mất tác dụng của cơ chế Docker Cache.
- **Lỗi Docker Build ngẫu nhiên:** Thường do đường truyền mạng quốc tế chậm chờn khi tải các base image hoặc kéo package (npm/pip/maven). Một nguyên nhân khác là sự thiếu đồng nhất về phiên bản package (không dùng lock file), khiến build lúc pass lúc fail.

5.2. Đề xuất 5 phương án Tối ưu hóa (Optimization Strategies)

1. Tối ưu Dockerfile & Tận dụng Docker Layer Cache:

- **Giải pháp:** Áp dụng Multi-stage build và sắp xếp thứ tự lệnh chuẩn xác, ví dụ như tách lệnh `COPY package.json` và `RUN npm`

`install` lên trước lệnh `COPY . .` (copy source code). Nhờ đó, nếu developer chỉ sửa code mà không thêm thư viện mới, Docker sẽ lấy ngay lớp cache của `npm install` (tiết kiệm hàng chục phút tải dependencies).

2. Triển khai Dependency Caching (Lưu trữ bộ đệm tĩnh):

- **Giải pháp:** Gắn kết (mount) một thư mục cache cục bộ cho các package manager trên Jenkins Agent (ví dụ: `~/ .npm` cho Node.js hoặc `~/ .m2` cho Maven). Kết hợp thiết lập Nexus Repository Manager nội bộ để làm Proxy/Mirror cache các thư viện, giúp tránh lỗi rớt mạng khi tải từ internet và tăng tốc độ kéo dependencies lên gấp nhiều lần.

3. Thực thi song song (Parallel Execution):

- **Giải pháp:** Sử dụng block `parallel` trong Jenkinsfile để chạy các bài test độc lập cùng lúc thay vì chờ nhau. Ví dụ: Chạy Unit Test, SonarQube (Code Quality) và Trivy (Security Scan) trên 3 luồng song song, giúp rút ngắn 1/3 thời gian của pha Testing.

4. Áp dụng Artifact Reuse (Xây dựng 1 lần, Triển khai mọi nơi):

- **Giải pháp:** Loại bỏ hoàn toàn việc "Build lại từ đầu" ở từng môi trường. Áp dụng chính xác mô hình mà bài báo cáo này đã cấu hình ở Task 1: Docker Image chỉ được Build ĐÚNG 1 LẦN sau khi pass Test. Sau đó, đẩy Artifact lên Registry và dùng Helm để tái sử dụng đúng Image Tag đó cho cả DEV, UAT và PROD.

5. Cấu hình giới hạn tài nguyên Agent (Resource Provisioning):

- **Giải pháp:** Khởi tạo Jenkins Agent động (Dynamic Agent) trên Kubernetes Pod thay vì dùng VM tĩnh, cấp phát đủ CPU/RAM (requests/limits) để tránh tình trạng Pipeline bị treo hoặc bị OOMKilled (Out of Memory) trong quá trình build các project lớn.

6. TASK 5: PRODUCTION INCIDENT RESPONSE

Ngữ cảnh: Sau khi release phiên bản mới, Response time tăng vọt, CPU Usage chạm đỉnh và người dùng bắt đầu phàn nàn về dịch vụ.

6.1. Quy trình Điều tra xác định Root Cause (Investigation)

Để tìm ra nguyên nhân gốc rễ một cách khoa học, quy trình điều tra cần áp dụng tam giác quan trắc (Observability):

1. **Metrics (Chỉ số đo lường):** Truy cập hệ thống giám sát **Prometheus & Grafana**. Kiểm tra các Dashboard để xác định chính xác Node/Pod nào đang bị quá tải CPU (CPU Throttling) và thời gian phản hồi (P99 Latency) tăng đột biến từ thời điểm deploy bản cập nhật.
2. **Tracing (Truy vết phân tán):** Sử dụng các công cụ APM hoặc OpenTelemetry (như Jaeger/Zipkin) để xem luồng đi của một request

(Spans). Việc này giúp khoanh vùng bottleneck đang nằm ở phía xử lý logic của App hay do chờ phản hồi chậm từ Database.

3. **Logs (Nhật ký hệ thống):** Truy vấn hệ thống Log tập trung (ELK Stack - Elasticsearch, Logstash, Kibana) hoặc dùng lệnh `kubectl logs` trên các Pod đang bị nghẽn để tìm kiếm các Exceptions, các vòng lặp vô hạn, hoặc câu truy vấn SQL thiếu index do bản code mới gây ra.

6.2. Xử lý tức thời: Rollback (Khôi phục dịch vụ)

- **No Hotfix in Prod:** Khi người dùng đang bị ảnh hưởng nghiêm trọng (Downtime/Degradation), ưu tiên tuyệt đối là khôi phục dịch vụ, **KHÔNG** ngồi debug và sửa code trực tiếp trên Production.
- **Hành động:** Kích hoạt ngay lập tức cơ chế Manual Rollback (Lớp bảo vệ thứ 2 đã trình bày ở Task 1). Sử dụng lệnh Helm: `helm rollback demo-app-production 0` để điều hướng Kubernetes tự động hạ cấp toàn bộ Pod về Revision ổn định trước đó. Thời gian phục hồi (MTTR) chỉ tính bằng giây. Sau khi hệ thống ổn định, team Dev mới bắt đầu debug trên môi trường Staging/DEV dựa trên các Logs đã thu thập.

6.3. Đề xuất Phòng tránh Sự cố tương tự (Preventive Measures)

1. **Thiết lập giới hạn tài nguyên (Resource Quotas & Limits):** Định nghĩa rõ ràng `resources.requests` và `resources.limits` cho CPU/RAM trong Helm Chart. Điều này ngăn chặn việc một Pod lỗi chiếm hết tài nguyên của Node, bảo vệ các dịch vụ khác trên cùng cụm Kubernetes.
2. **Kịch bản Triển khai an toàn (Canary / Blue-Green Deployment):** Thay vì thay thế 100% bản cũ cùng lúc (Rolling Update), áp dụng cấu hình Ingress để chuyển trước 5% lượng traffic thật của người dùng vào phiên bản mới (Canary). Nếu Metrics giám sát báo lỗi CPU/Latency, tự động Rollback ngay ở mức 5% này để giảm thiểu ảnh hưởng.
3. **Tích hợp Load Testing vào CI/CD:** Thêm một Stage `Performance Test` (sử dụng công cụ K6 hoặc JMeter) vào Pipeline tại môi trường UAT/Staging. Hệ thống phải chịu tải vượt mức Baseline thông thường trước khi nút `Approve for PROD` được hiển thị.